

Given two patterns P and P' , describe how to construct a finite automaton that determines all occurrences of *either* pattern. Try to minimize the number of states in your automaton.

32.3-6

Given a pattern P containing gap characters (see Exercise 32.1-4), show how to build a finite automaton that can find an occurrence of P in a text T in $O(n)$ matching time, where $n = |T|$.

★ 32.4 The Knuth-Morris-Pratt algorithm

Knuth, Morris, and Pratt developed a linear-time string matching algorithm that avoids computing the transition function δ altogether. Instead, the KMP algorithm uses an auxiliary function π , which it precomputes from the pattern in $\Theta(m)$ time and stores in an array $\pi[1:m]$. The array π allows the algorithm to compute the transition function δ efficiently (in an amortized sense) “on the fly” as needed. Loosely speaking, for any state $q = 0, 1, \dots, m$ and any character $a \in \Sigma$, the value $\pi[q]$ contains the information needed to compute $\delta(q, a)$ but that does not depend on a . Since the array π has only m entries, whereas δ has $\Theta(m |\Sigma|)$ entries, the KMP algorithm saves a factor of $|\Sigma|$ in the preprocessing time by computing π rather than δ . Like the procedure `FINITE-AUTOMATON-MATCHER`, once preprocessing has completed, the KMP algorithm uses $\Theta(n)$ matching time.

The prefix function for a pattern

The prefix function π for a pattern encapsulates knowledge about how the pattern matches against shifts of itself. The KMP algorithm takes advantage of this information to avoid testing useless shifts in the naive pattern-matching algorithm and to avoid precomputing the full transition function δ for a string-matching automaton.

Consider the operation of the naive string matcher. Figure 32.9(a) shows a particular shift s of a template containing the pattern $P = \text{ababaca}$ against a text T . For this example, $q = 5$ of the characters have matched successfully, but the 6th pattern character fails to match

the corresponding text character. The information that q characters have matched successfully determines the corresponding text characters. Because these q text characters match, certain shifts must be invalid. In the example of the figure, the shift $s + 1$ is necessarily invalid, since the first pattern character (a) would be aligned with a text character that does not match the first pattern character, but does match the second pattern character (b). The shift $s' = s + 2$ shown in part (b) of the figure, however, aligns the first three pattern characters with three text characters that necessarily match.

More generally, suppose that you know that $P[:q] \sqsupseteq T[s + q]$ or, equivalently, that $P[1:q] = T[s + 1:s + q]$. You want to shift P so that some shorter prefix $P[:k]$ of P matches a suffix of $T[s + q]$, if possible. You might have more than one choice for how much to shift, however. In Figure 32.9(b), shifting P by 2 positions works, so that $P[:3] \sqsupseteq T[s + q]$, but so does shifting P by 4 positions, so that $P[:1] \sqsupseteq T[s + q]$ in Figure 32.9(c). If more than one shift amount works, you should choose the smallest shift amount so that you do not miss any potential matches. Put more precisely, you want to answer this question:

Given that pattern characters $P[1:q]$ match text characters $T[s + 1:s + q]$ (that is, $P[:q] \sqsupseteq T[s + q]$), what is the least shift $s' > s$ such that for some $k < q$,

$$P[1:k] = T[s' + 1:s' + k], \quad (32.6)$$

(that is, $P[:k] \sqsupseteq T[s' + k]$), where $s' + k = s + q$?

Here's another way to look at this question. If you know $P[:q] \sqsupseteq T[s + q]$, then how do you find the longest proper prefix $P[:k]$ of $P[:q]$ that is also a suffix of $T[s + q]$? These questions are equivalent because given s and q , requiring $s' + k = s + q$ means that finding the smallest shift s' (2 in Figure 32.9(b)) is tantamount to finding the longest prefix length k (3 in Figure 32.9(b)). If you add the difference $q - k$ in the lengths of these prefixes of P to the shift s , you get the new shift s' , so that $s' = s + (q - k)$. In the best case, $k = 0$, so that $s' = s + q$, immediately ruling out shifts $s + 1, s + 2, \dots, s + q - 1$. In any case, at the new shift s' , it is redundant to compare the first k characters of P with the

corresponding characters of T , since equation (32.6) guarantees that they match.

As Figure 32.9(d) demonstrates, you can precompute the necessary information by comparing the pattern against itself. Since $T[s' + 1:s' + k]$ is part of the matched portion of the text, it is a suffix of the string $P[:q]$. Therefore, think of equation (32.6) as asking for the greatest $k < q$ such that $P[:k] \sqsupseteq P[:q]$. Then, the new shift $s' = s + (q - k)$ is the next potentially valid shift. It will be convenient to store, for each value of q , the number k of matching characters at the new shift s' , rather than storing, say, the amount $s' - s$ to shift by.

Let's look at the precomputed information a little more formally. For a given pattern $P[1:m]$, the *prefix function* for P is the function $\pi : \{1, 2, \dots, m\} \rightarrow \{0, 1, \dots, m - 1\}$ such that

$$\pi[q] = \max\{k : k < q \text{ and } P[:k] \sqsupseteq P[:q]\}.$$

That is, $\pi[q]$ is the length of the longest prefix of P that is a proper suffix of $P[:q]$. Here is the complete prefix function π for the pattern `ababaca`:

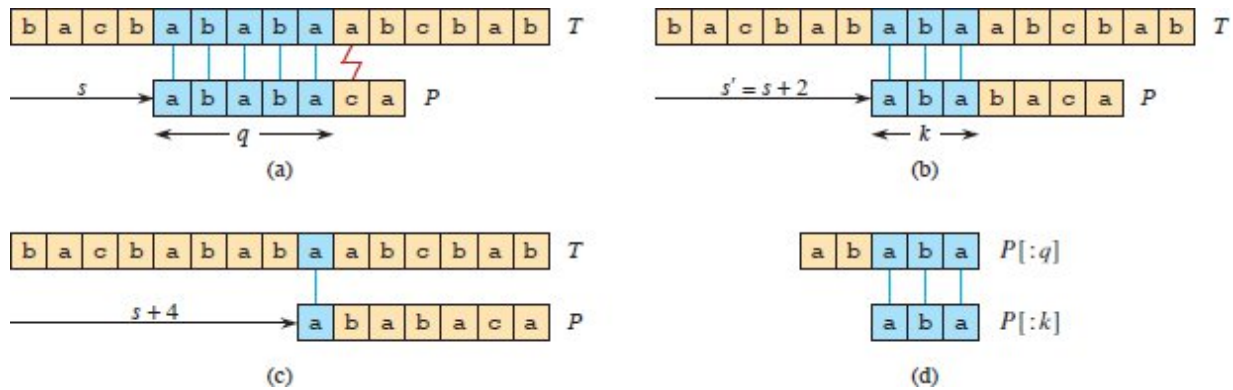


Figure 32.9 The prefix function π . **(a)** The pattern $P = ababaca$ aligns with a text T so that the first $q = 5$ characters match. Matching characters, in blue, are connected by blue lines. **(b)** Knowing these particular 5 matched characters ($P[:5]$) suffices to deduce that a shift of $s + 1$ is invalid, but that a shift of $s' = s + 2$ is consistent with everything known about the text and therefore is potentially valid. The prefix $P[:k]$, where $k = 3$, aligns with the text seen so far. **(c)** A shift of $s + 4$ is also potentially valid, but it leaves only the prefix $P[:1]$ aligned with the text seen so far. **(d)** To precompute useful information for such deductions, compare the pattern with itself. Here, the longest prefix of P that is also a proper suffix of $P[:5]$ is $P[:3]$. The array π represents this precomputed information, so that $\pi[5] = 3$. Given that q characters have matched successfully at shift s , the next potentially valid shift is at $s' = s + (q - \pi[q])$ as shown in part (b).

i	1	2	3	4	5	6	7
$P[i]$	a	b	a	b	a	c	a
$\pi[i]$	0	0	1	2	3	0	1

The procedure **KMP-MATCHER** on the following page gives the Knuth-Morris-Pratt matching algorithm. The procedure follows from **FINITE-AUTOMATON-MATCHER** for the most part. To compute π , **KMP-MATCHER** calls the auxiliary procedure **COMPUTE-PREFIX-FUNCTION**. These two procedures have much in common, because both match a string against the pattern P : **KMP-MATCHER** matches the text T against P , and **COMPUTE-PREFIX-FUNCTION** matches P against itself.

Next, let's analyze the running times of these procedures. Then we'll prove them correct, which will be more complicated.

Running-time analysis

The running time of COMPUTE-PREFIX-FUNCTION is $\Theta(m)$, which we show by using the aggregate method of amortized analysis (see Section 16.1). The only tricky part is showing that the **while** loop of lines 5–6 executes $O(m)$ times altogether. Starting with some observations about k , we'll show that it makes at most $m-1$ iterations. First, line 3 starts k at 0, and the only way that k increases is by the increment operation in line 8, which executes at most once per iteration of the **for** loop of lines 4–9. Thus, the total increase in k is at most $m-1$. Second, since $k < q$ upon entering the **for** loop and each iteration of the loop increments q , we always have $k < q$. Therefore, the assignments in lines 2 and 9 ensure that $\pi[q] < q$ for all $q = 1, 2, \dots, m$, which means that each iteration of the **while** loop decreases k . Third, k never becomes negative. Putting these facts together, we see that the total decrease in k from the **while** loop is bounded from above by the total increase in k over all iterations of the **for** loop, which is $m - 1$. Thus, the **while** loop iterates at most $m - 1$ times in all, and COMPUTE-PREFIX-FUNCTION runs in $\Theta(m)$ time.

KMP-MATCHER(T, P, n, m)

```

1  $\pi$  = COMPUTE-PREFIX-FUNCTION( $P, m$ )
2  $q = 0$                                 // number of characters matched
3 for  $i = 1$  to  $n$                         // scan the text from left to right
4   while  $q > 0$  and  $P[q + 1] \neq T[i]$ 
5      $q = \pi[q]$                           // next character does not match
6   if  $P[q + 1] == T[i]$ 
7      $q = q + 1$                           // next character matches
8   if  $q == m$                             // is all of  $P$  matched?
9     print "Pattern occurs with shift"  $i - m$ 
10     $q = \pi[q]$                           // look for the next match
```

COMPUTE-PREFIX-FUNCTION(P, m)

```

1 let  $\pi[1:m]$  be a new array
2  $\pi[1] = 0$ 
3  $k = 0$ 
4 for  $q = 2$  to  $m$ 
```

```

5   while  $k > 0$  and  $P[k + 1] \neq P[q]$ 
6        $k = \pi[k]$ 
7   if  $P[k + 1] == P[q]$ 
8        $k = k + 1$ 
9    $\pi[q] = k$ 
10 return  $\pi$ 

```

Exercise 32.4-4 asks you to show, by a similar aggregate analysis, that the matching time of KMP-MATCHER is $\Theta(n)$.

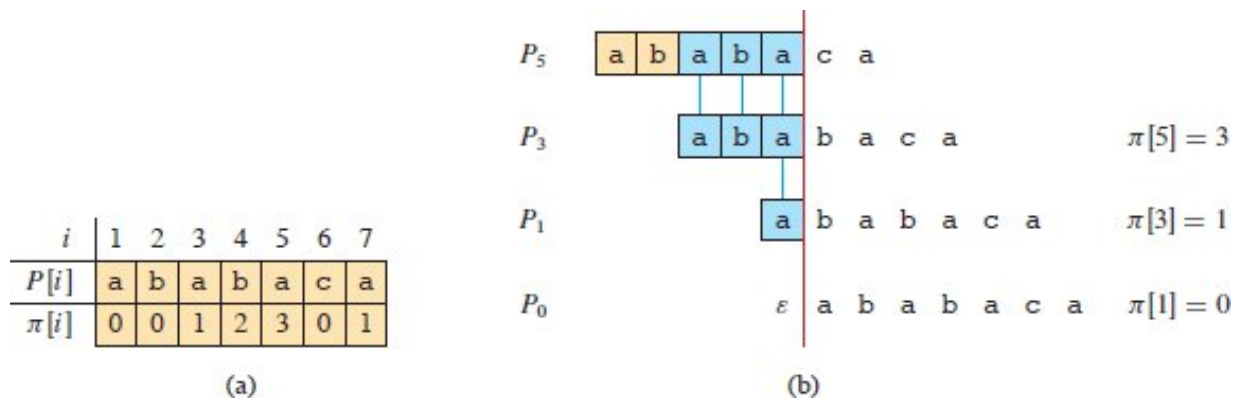


Figure 32.10 An illustration of Lemma 32.5 for the pattern $P = ababaca$ and $q = 5$. **(a)** The π function for the given pattern. Since $\pi[5] = 3$, $\pi[3] = 1$, and $\pi[1] = 0$, iterating π gives $\pi^*[5] = \{3, 1, 0\}$. **(b)** Sliding the template containing the pattern P to the right and noting when some prefix $P[:k]$ of P matches up with some proper suffix of $P[:5]$. Matches occur when $k = 3, 1$, and 0 . In the figure, the first row gives P , and the vertical red line is drawn just after $P[5]$. Successive rows show all the shifts of P that cause some prefix $P[:k]$ of P to match some suffix of $P[:5]$. Successfully matched characters are shown in blue. Blue lines connect aligned matching characters. Thus, $\{k : k < 5 \text{ and } P[:k] \sqsupseteq P[:5]\} = \{3, 1, 0\}$. Lemma 32.5 claims that $\pi^*[q] = \{k : k < q \text{ and } P[:k] \sqsupseteq P[:q]\}$ for all q .

Compared with FINITE-AUTOMATON-MATCHER, by using π rather than δ , the KMP algorithm reduces the time for preprocessing the pattern from $O(m |\Sigma|)$ to $\Theta(m)$, while keeping the actual matching time bounded by $\Theta(n)$.

Correctness of the prefix-function computation

We'll see a little later that the prefix function π helps to simulate the transition function δ in a string-matching automaton. But first, we need

to prove that the procedure COMPUTE-PREFIX-FUNCTION does indeed compute the prefix function correctly. Doing so requires finding all prefixes $P[:k]$ that are proper suffixes of a given prefix $P[:q]$. The value of $\pi[q]$ gives us the length of the longest such prefix, but the following lemma, illustrated in Figure 32.10, shows that iterating the prefix function π generates all the prefixes $P[:k]$ that are proper suffixes of $P[:q]$. Let

$$\pi^*[q] = \{\pi[q], \pi^{(2)}[q], \pi^{(3)}[q], \dots, \pi^{(t)}[q]\},$$

where $\pi^{(i)}[q]$ is defined in terms of functional iteration, so that $\pi^{(0)}[q] = q$ and $\pi^{(i)}[q] = \pi[\pi^{(i-1)}[q]]$ for $i \geq 1$ (so that $\pi[q] = \pi^{(1)}[q]$), and where the sequence in $\pi^*[q]$ stops upon reaching $\pi^{(t)}[q] = 0$ for some $t \geq 1$.

Lemma 32.5 (Prefix-function iteration lemma)

Let P be a pattern of length m with prefix function π . Then, for $q = 1, 2, \dots, m$, we have $\pi^*[q] = \{k : k < q \text{ and } P[:k] \sqsupset P[:q]\}$.

Proof We first prove that $\pi^*[q] \subseteq \{k : k < q \text{ and } P[:k] \sqsupset P[:q]\}$ or, equivalently,

$$i \in \pi^*[q] \text{ implies } P[:i] \sqsupset P[:q]. \quad (32.7)$$

If $i \in \pi^*[q]$, then $i = \pi^{(u)}[q]$ for some $u > 0$. We prove equation (32.7) by induction on u . For $u = 1$, we have $i = \pi[q]$, and the claim follows since $i < q$ and $P[:\pi[q]] \sqsupset P[:q]$ by the definition of π . Now consider some $u \geq 1$ such that both $\pi^{(u)}[q]$ and $\pi^{(u+1)}[q]$ belong to $\pi^*[q]$. Let $i = \pi^{(u)}[q]$, so that $\pi[i] = \pi^{(u+1)}[q]$. The inductive hypothesis is that $P[:i] \sqsupset P[:q]$. Because the relations $<$ and $\sqsupset$ are transitive, we have $\pi[i] < i < q$ and $P[:\pi[i]] \sqsupset P[:i] \sqsupset P[:q]$, which establishes equation (32.7) for all i in $\pi^*[q]$. Therefore, $\pi^*[q] \subseteq \{k : k < q \text{ and } P[:k] \sqsupset P[:q]\}$.

We now prove that $\{k : k < q \text{ and } P[:k] \sqsupset P[:q]\} \subseteq \pi^*[q]$ by contradiction. Suppose to the contrary that the set $\{k : k < q \text{ and } P[:k] \sqsupset P[:q]\} - \pi^*[q]$ is nonempty, and let j be the largest number in the set. Because $\pi[q]$ is the largest value in $\{k : k < q \text{ and } P[:k] \sqsupset P[:q]\}$ and $\pi^*[q] \in \pi^*[q]$, it must be the case that $j < [q]$. Having established that $\pi^*[q]$

contains at least one integer greater than j , let j' denote the smallest such integer. (We can choose $j' = \pi[q]$ if no other number in $\pi^*[q]$ is greater than j .) We have $P[:j] \sqsupset P[:q]$ because $j \in \{k : k < q \text{ and } P[:k] \sqsupset P[:q]\}$, and from $j' \in \pi^*[q]$ and equation (32.7), we have $P[:j'] \sqsupset P[:q]$. Thus, $P[:j] \sqsupset P[:j']$ by Lemma 32.1, and j is the largest value less than j' with this property. Therefore, we must have $\pi[j'] = j$ and, since $j' \in \pi^*[q]$, we must have $j \in \pi^*[q]$ as well. This contradiction proves the lemma. ■

The algorithm COMPUTE-PREFIX-FUNCTION computes $\pi[q]$, in order, for $q = 1, 2, \dots, m$. Setting $\pi[1]$ to 0 in line 2 of COMPUTE-PREFIX-FUNCTION is certainly correct, since $\pi[q] < q$ for all q . We'll use the following lemma and its corollary to prove that COMPUTE-PREFIX-FUNCTION computes $\pi[q]$ correctly for $q > 1$.

Lemma 32.6

Let P be a pattern of length m , and let π be the prefix function for P . For $q = 1, 2, \dots, m$, if $\pi[q] > 0$, then $\pi[q] - 1 \in \pi^*[q - 1]$.

Proof Let $r = \pi[q] > 0$, so that $r < q$ and $P[:r] \sqsupset P[:q]$, and thus, $r - 1 < q - 1$ and $P[:r - 1] \sqsupset P[:q - 1]$ (by dropping the last character from $P[:r]$ and $P[:q]$, which we can do because $r > 0$). By Lemma 32.5, therefore, $r - 1 \in \pi^*[q - 1]$. Thus, we have $\pi[q] - 1 = r - 1 \in \pi^*[q - 1]$. ■

For $q = 2, 3, \dots, m$, define the subset $E_{q-1} \subseteq \pi^*[q - 1]$ by

$$\begin{aligned} E_{q-1} &= \{k \in \pi^*[q - 1] : P[k + 1] = P[q]\} \\ &= \{k : k < q - 1 \text{ and } P[:k] \sqsupset P[:q - 1] \text{ and } P[k + 1] = P[q]\} \\ &\quad \text{(by Lemma 32.5)} \\ &= \{k : k < q - 1 \text{ and } P[:k + 1] \sqsupset P[:q]\}. \end{aligned}$$

The set E_{q-1} consists of the values $k < q - 1$ for which $P[:k] \sqsupset P[:q - 1]$ and for which, because $P[k + 1] = P[q]$, we have $P[:k + 1] \sqsupset P[:q]$. Thus, E_{q-1} consists of those values $k \in \pi^*[q - 1]$ such that extending $P[:k]$ to $P[:k + 1]$ produces a proper suffix of $P[:q]$.

Corollary 32.7

Let P be a pattern of length m , and let π be the prefix function for P . Then, for $q = 2, 3, \dots, m$,

$$\pi[q] = \begin{cases} 0 & \text{if } E_{q-1} = \emptyset, \\ 1 + \max E_{q-1} & \text{if } E_{q-1} \neq \emptyset. \end{cases}$$

Proof If E_{q-1} is empty, there is no $k \in \pi^*[q-1]$ (including $k = 0$) such that extending $P[:k]$ to $P[:k+1]$ produces a proper suffix of $P[:q]$. Therefore, $\pi[q] = 0$.

If, instead, E_{q-1} is nonempty, then for each $k \in E_{q-1}$, we have $k+1 < q$ and $P[:k+1] \sqsupset P[:q]$. Therefore, the definition of $\pi[q]$ gives

$$\pi[q] \geq 1 + \max E_{q-1}. \quad (32.8)$$

Note that $\pi[q] > 0$. Let $r = \pi[q] - 1$, so that $r+1 = \pi[q] > 0$, and therefore $P[:r+1] \sqsupset P[:q]$. If a nonempty string is a suffix of another, then the two strings must have the same last character. Since $r+1 > 0$, the prefix $P[:r+1]$ is nonempty, and so $P[r+1] = P[q]$. Furthermore, $r \in \pi^*[q-1]$ by Lemma 32.6. Therefore, $r \in E_{q-1}$, and so $\pi[q] - 1 = r \leq \max E_{q-1}$ or, equivalently,

$$\pi[q] \leq 1 + \max E_{q-1}. \quad (32.9)$$

Combining equations (32.8) and (32.9) completes the proof. ■

We now finish the proof that COMPUTE-PREFIX-FUNCTION computes π correctly. The key is to combine the definition of E_{q-1} with the statement of Corollary 32.7, so that $\pi[q]$ equals 1 plus the greatest value of k in $\pi^*[q-1]$ such that $P[k+1] = P[q]$. First, in COMPUTE-PREFIX-FUNCTION, $k = \pi[q-1]$ at the start of each iteration of the **for** loop of lines 4–9. This condition is enforced by lines 2 and 3 when the loop is first entered, and it remains true in each successive iteration because of line 9. Lines 5–8 adjust k so that it becomes the correct value of $\pi[q]$. The **while** loop of lines 5–6 searches through all values $k \in \pi^*[q-1]$ in decreasing order to find the value of $\pi[q]$. The loop terminates either because k reaches 0 or $P[k+1] = P[q]$. Because the “and”

operator short-circuits, if the loop terminates because $P[k + 1] = P[q]$, then k must have also been positive, and so k is the greatest value in E_{q-1} . In this case, lines 7–9 set $\pi[q]$ to $k + 1$, according to Corollary 32.7. If, instead, the **while** loop terminates because $k = 0$, then there are two possibilities. If $P[1] = P[q]$, then $E_{q-1} = \{0\}$, and lines 7–9 set both k and $\pi[q]$ to 1. If $k = 0$ and $P[1] \neq P[q]$, however, then $E_{q-1} = \emptyset$. In this case, line 9 sets $\pi[q]$ to 0, again according to Corollary 32.7, which completes the proof of the correctness of COMPUTE-PREFIX-FUNCTION.

Correctness of the Knuth-Morris-Pratt algorithm

You can think of the procedure KMP-MATCHER as a reimplemented version of the procedure FINITE-AUTOMATON-MATCHER, but using the prefix function π to compute state transitions. Specifically, we'll prove that in the i th iteration of the **for** loops of both KMP-MATCHER and FINITE-AUTOMATON-MATCHER, the state q has the same value upon testing for equality with m (at line 8 in KMP-MATCHER and at line 4 in FINITE-AUTOMATON-MATCHER). Once we have argued that KMP-MATCHER simulates the behavior of FINITE-AUTOMATON-MATCHER, the correctness of KMP-MATCHER follows from the correctness of FINITE-AUTOMATON-MATCHER (though we'll see a little later why line 10 in KMP-MATCHER is necessary).

Before formally proving that KMP-MATCHER correctly simulates FINITE-AUTOMATON-MATCHER, let's take a moment to understand how the prefix function π replaces the δ transition function. Recall that when a string-matching automaton is in state q and it scans a character $a = T[i]$, it moves to a new state $\delta(q, a)$. If $a = P[q + 1]$, so that a continues to match the pattern, then the state number is incremented: $\delta(q, a) = q + 1$. Otherwise, $a \neq P[q + 1]$, so that a does not continue to match the pattern, and the state number does not increase: $0 \leq \delta(q, a) \leq q$. In the first case, when a continues to match, KMP-MATCHER moves to state $q + 1$ without referring to the π function:

the **while** loop test in line 4 immediately comes up false, the test in line 6 comes up true, and line 7 increments q .

The π function comes into play when the character a does not continue to match the pattern, so that the new state $\delta(q, a)$ is either q or to the left of q along the spine of the automaton. The **while** loop of lines 4–5 in KMP-MATCHER iterates through the states in $\pi^*[q]$, stopping either when it arrives in a state, say q' , such that a matches $P[q' + 1]$ or q' has gone all the way down to 0. If a matches $P[q' + 1]$, then line 7 sets the new state to $q' + 1$, which should equal $\delta(q, a)$ for the simulation to work correctly. In other words, the new state $\delta(q, a)$ should be either state 0 or a state numbered 1 more than some state in $\pi^*[q]$.

Let's look at the example in Figures 32.6 and 32.10, which are for the pattern $P = \text{ababaca}$. Suppose that the automaton is in state $q = 5$, having matched ababa . The states in $\pi^*[5]$ are, in descending order, 3, 1, and 0. If the next character scanned is c , then you can see that the automaton moves to state $\delta(5, c) = 6$ in both FINITE-AUTOMATON-MATCHER (line 3) and KMP-MATCHER (line 7). Now suppose that the next character scanned is instead b , so that the automaton should move to state $\delta(5, b) = 4$. The **while** loop in KMP-MATCHER exits after executing line 5 once, and the automaton arrives in state $q' = \pi[5] = 3$. Since $P[q' + 1] = P[4] = b$, the test in line 6 comes up true, and the automaton moves to the new state $q' + 1 = 4 = \delta(5, b)$. Finally, suppose that the next character scanned is instead a , so that the automaton should move to state $\delta(5, a) = 1$. The first three times that the test in line 4 executes, the test comes up true. The first time finds that $P[6] = c \neq a$, and the automaton moves to state $\pi[5] = 3$ (the first state in $\pi^*[5]$). The second time finds that $P[4] = b \neq a$, and the automaton moves to state $\pi[3] = 1$ (the second state in $\pi^*[5]$). The third time finds that $P[2] = b \neq a$, and the automaton moves to state $\pi[1] = 0$ (the last state in $\pi^*[5]$). The **while** loop exits once it arrives in state $q' = 0$. Now line 6 finds that $P[q' + 1] = P[1] = a$, and line 7 moves the automaton to the new state $q' + 1 = 1 = \delta(5, a)$.

Thus, the intuition is that KMP-MATCHER iterates through the states in $\pi^*[q]$ in decreasing order, stopping at some state q' and then possibly moving to state $q' + 1$. Although that might seem like a lot of

work just to simulate computing $\delta(q, a)$, bear in mind that asymptotically, KMP-MATCHER is no slower than FINITE-AUTOMATON-MATCHER.

We are now ready to formally prove the correctness of the Knuth-Morris-Pratt algorithm. By Theorem 32.4, we have that $q = \sigma(T[:i])$ after each time line 3 of FINITE-AUTOMATON-MATCHER executes. Therefore, it suffices to show that the same property holds with regard to the **for** loop in KMP-MATCHER. The proof proceeds by induction on the number of loop iterations. Initially, both procedures set q to 0 as they enter their respective **for** loops for the first time. Consider iteration i of the **for** loop in KMP-MATCHER. By the inductive hypothesis, the state number q equals $\sigma(T[:i-1])$ at the start of the loop iteration. We need to show that when line 8 is reached, the new value of q is $\sigma(T[:i])$. (Again, we'll handle line 10 separately.)

Considering q to be the state number at the start of the **for** loop iteration, when KMP-MATCHER considers the character $T[i]$, the longest prefix of P that is a suffix of $T[:i]$ is either $P[:q+1]$ (if $P[q+1] = T[i]$) or some prefix (not necessarily proper, and possibly empty) of $P[:q]$. We consider separately the three cases in which $\sigma(T[:i]) = 0$, $\sigma(T[:i]) = q+1$, and $0 < \sigma(T[:i]) \leq q$.

- If $\sigma(T[:i]) = 0$, then $P[:0] = \epsilon$ is the only prefix of P that is a suffix of $T[:i]$. The **while** loop of lines 4–5 iterates through each value q' in $\pi^*[q]$, but although $P[:q'] \supseteq P[:q] \supseteq T[:i-1]$ for every $q' \in \pi^*[q]$ (because $<$ are $\supseteq$ are transitive relations), the loop never finds a q' such that $P[q'+1] = T[i]$. The loop terminates when q reaches 0, and of course line 7 does not execute. Therefore, $q = 0$ at line 8, so that now $q = \sigma(T[:i])$.
- If $\sigma(T[:i]) = q+1$, then $P[q+1] = T[i]$, and the **while** loop test in line 4 fails the first time through. Line 7 executes, incrementing the state number to $q+1$, which equals $\sigma(T[:i])$.
- If $0 < \sigma(T[:i]) \leq q'$, then the **while** loop of lines 4–5 iterates at least once, checking in decreasing order each value in $\pi^*[q]$ until it stops at some $q' < q$. Thus, $P[:q']$ is the longest prefix of $P[:q]$ for which $P[q'+1] = T[i]$, so that when the **while** loop terminates, $q' +$

$1 = \sigma(P[:q]T[i])$. Since $q = \sigma(T[:i-1])$, Lemma 32.3 implies that $\sigma(T[:i-1]T[i]) = \sigma(P[:q]T[i])$. Thus we have

$$\begin{aligned} q' + 1 &= \sigma(P[:q]T[i]) \\ &= \sigma(T[:i-1]T[i]) \\ &= \sigma(T[:i]) \end{aligned}$$

when the **while** loop terminates. After line 7 increments q , the new state number q equals $\sigma(T[:i])$.

Line 10 is necessary in KMP-MATCHER, because otherwise, line 4 might try to reference $P[m+1]$ after finding an occurrence of P . (The argument that $q = \sigma(T[:i-1])$ upon the next execution of line 4 remains valid by the hint given in Exercise 32.4-8: that $\delta(m, a) = \delta(\pi[m], a)$ or, equivalently, $\sigma(Pa) = \sigma(P[:\pi[m]]a)$ for any $a \in \Sigma$.) The remaining argument for the correctness of the Knuth-Morris-Pratt algorithm follows from the correctness of FINITE-AUTOMATON-MATCHER, since we have shown that KMP-MATCHER simulates the behavior of FINITE-AUTOMATON-MATCHER.

Exercises

32.4-1

Compute the prefix function π for the pattern ababbabbabbababbabb.

32.4-2

Give an upper bound on the size of $\pi^*[q]$ as a function of q . Give an example to show that your bound is tight.

32.4-3

Explain how to determine the occurrences of pattern P in the text T by examining the π function for the string PT (the string of length $m+n$ that is the concatenation of P and T).

32.4-4

Use an aggregate analysis to show that the running time of KMP-MATCHER is $\Theta(n)$.

32.4-5

Use a potential function to show that the running time of KMP-MATCHER is $\Theta(n)$.

32.4-6

Show how to improve KMP-MATCHER by replacing the occurrence of π in line 5 (but not line 10) by π' , where π' is defined recursively for $q = 1, 2, \dots, m - 1$ by the equation

$$\pi'[q] = \begin{cases} 0 & \text{if } \pi[q] = 0, \\ \pi'[\pi[q]] & \text{if } \pi[q] \neq 0 \text{ and } P[\pi[q] + 1] = P[q + 1], \\ \pi[q] & \text{if } \pi[q] \neq 0 \text{ and } P[\pi[q] + 1] \neq P[q + 1]. \end{cases}$$

Explain why the modified algorithm is correct, and explain in what sense this change constitutes an improvement.

32.4-7

Give a linear-time algorithm to determine whether a text T is a cyclic rotation of another string T' . For example, `braze` and `zebra` are cyclic rotations of each other.

★ 32.4-8

Give an $O(m|\Sigma|)$ -time algorithm for computing the transition function δ for the string-matching automaton corresponding to a given pattern P . (*Hint: Prove that $\delta(q, a) = \delta(\pi[q], a)$ if $q = m$ or $P[q + 1] \neq a$.*)

32.5 Suffix arrays

The algorithms we have seen thus far in this chapter can efficiently find all occurrences of a pattern in a text. That is, however, all they can do. This section presents a different approach—suffix arrays—with which you can find all occurrences of a pattern in a text, but also quite a bit more. A suffix array won't find all occurrences of a pattern as quickly as, say, the Knuth-Morris-Pratt algorithm, but its additional flexibility makes it well worth studying.